Quota Groups Management – Component Guide & E2E Test Approach

Nov 12, 2026 · @prem bommidi

What the component does

Quota Groups Management lets operators pool Azure compute quota across subscriptions and move it to where it is needed, instead of raising a support ticket for every subscription, VM size and region.

It lives in `Azure-Platform/components/quota-groups-management/` and was built recently by the DevOps team. This guide is based on a Copilot walkthrough of the checked-in code; confirm details with the component's author.

It does four jobs:

1. **Create or remove a quota group** under a management group. It refuses to remove a group that still has members.
2. **Manage members**: remove subscriptions from a group, and sync Enterprise Agreement subscriptions into it.
3. **Move quota** to a subscription that needs more (`Request-AzureQuota` , the main feature).
4. **Read quota**: list VM size (SKU) names and show a subscription's current allocation.

Key concepts

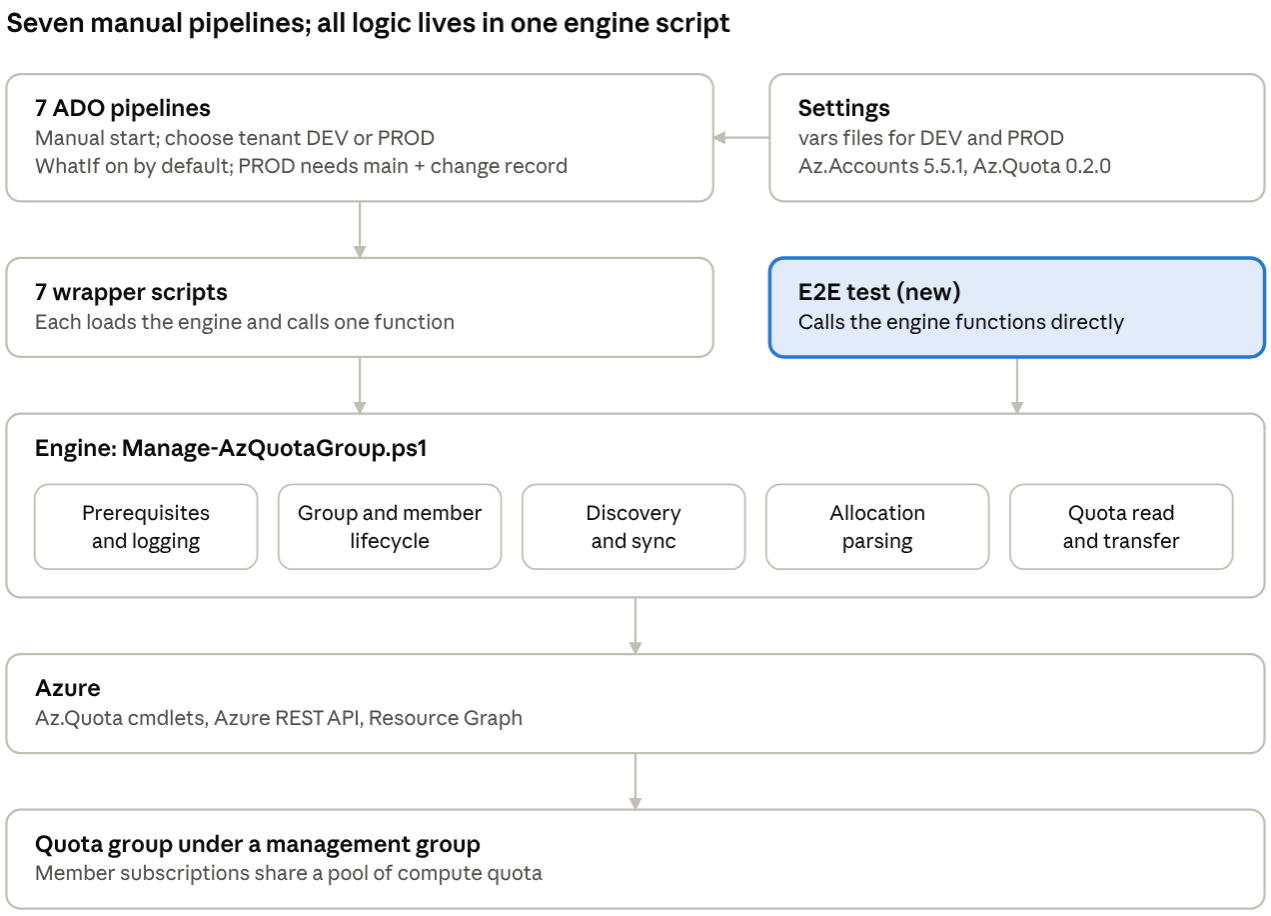
A quota group is a shared pot of compute quota: member subscriptions can give spare quota to it, and a member that needs more can take from it.

Term	Meaning
Quota	The maximum vCPUs a subscription may use for one VM family (SKU) in one region
Quota group	The shared pot, created under a management group and identified by `ManagementGroupId` + `QuotaGroupName`
Member	A subscription that has joined the group
Recipient	The member that needs more quota

Term	Meaning
Donor	A member with spare quota that can give some away
Increment	`Request-AzureQuota` asks for an additional amount, not a final total; the code rounds it up to a multiple of 10
WhatIf	Preview mode: logs the planned changes and changes nothing; the default in the pipelines
Excluded APMIDs	Applications whose subscriptions must never be used as donors (read from an exclusion file)

How it is built

The component has three layers: pipelines start a run, thin wrapper scripts pass the inputs, and one engine script does all the work against Azure.

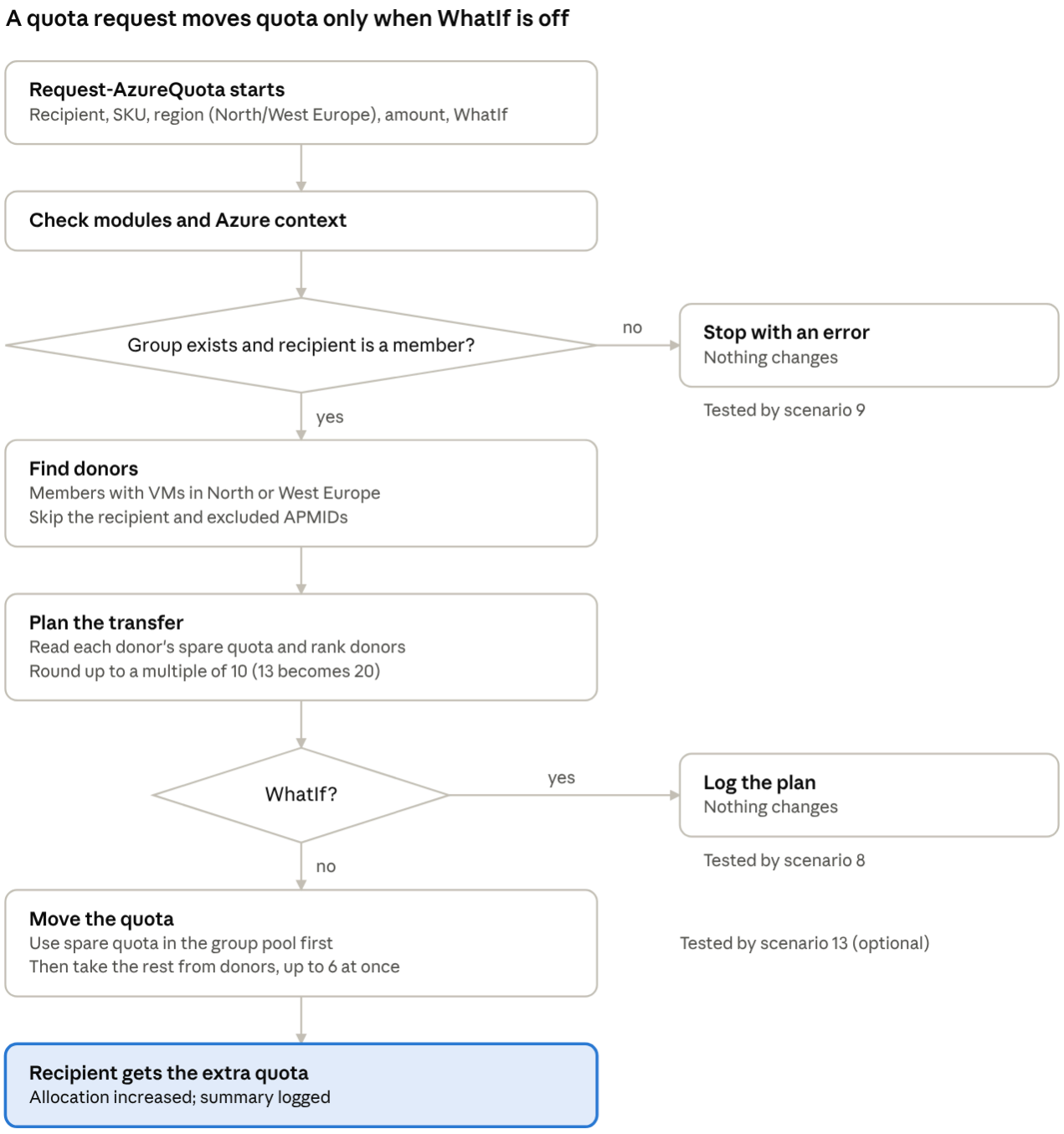


component structure · pipelines → wrappers → engine → Azure

Settings and pinned module versions feed every pipeline. The new E2E test skips the pipelines and wrappers and calls the engine's functions directly.

How a quota request works

`Request-AzureQuota` is the main feature: it gives a recipient subscription more quota by taking spare quota from other members of the group.



quota request flow · 2 checks, 1 preview branch

With WhatIf on (the pipeline default), the run stops after planning and changes nothing. That is why the E2E test can safely check the plan (scenario 8) before anyone approves a real transfer (scenario 13).

Pipelines, environments and safety

All seven pipelines are manual only (`trigger: none`), and no schedules are checked in. Each one calls one wrapper script.

Pipeline	Wrapper script	What it does
`pipeline_create_quota_group.yml`	`Initialize-AzureQuotaGroup.ps1`	Creates the group if it doesn't exist
`pipeline_remove_quota_group.yml`	`Remove-AzureQuotaGroup.ps1`	Deletes the group, only if it has no members
`pipeline_remove_subscription_from_quota_group.yml`	`Remove-AzureQuotaGroupSubscription.ps1`	Removes member subscriptions (comma-separated IDs)
`pipeline_request_quota.yml`	`Request-AzureQuota.ps1`	Moves quota to a recipient in North or West Europe
`pipeline_get_subscription_quota.yml`	`Get-AzureQuotaGroupSubscriptionQuota.ps1`	Reads a member's allocation, with a classic-quota fallback
Sync pipeline (file name to confirm)	`Sync-AzureQuotaGroupProdSubscriptions.ps1`	Syncs Enterprise Agreement subscriptions into the group; has DEV and PROD stages despite its name
Quota names pipeline (file name to confirm)	`Get-AzureComputeQuotaNames.ps1`	Lists SKU names and their display names

Safety rules built in:

- Every run needs a manual start and a chosen `tenant`: DEV or PROD.
- PROD stages also require the `main` branch and include a change-registration task (`SwissReMultistageAuditorExp@0`).
- `WhatIf` is the default, so a run only previews unless someone turns it off.
- Errors stop the run: `Write-Log -Level ERROR` throws.

Settings come from environment files: `environments/swissredev_com/exp/vars.yaml` (DEV) and `environments/swissre_com/prod/vars.yml` (PROD). They hold the group name, management group and resource provider.

Module versions are pinned in `build-meta.jsonc` (Az.Accounts 5.5.1, Az.Quota 0.2.0) and installed by `pipelines/templates/install-azquota-module.yml`.

Existing tests and the gap

The component has unit tests only. Nothing checks that it works against real Azure, and the quota-moving logic isn't tested at all.

	Exists today	Not covered
Unit tests	One Pester file, `scripts/Manage-AzQuotaGroup.Unit.Tests.ps1`, with Azure calls mocked	Full donor rebalancing: it runs in parallel and uses Resource Graph, so the test comment says it isn't unit-tested
What the unit tests check	Logging, prerequisites, group create/delete, membership, quota parsing and updates, subscription discovery, REST member parsing, syncing, early request validation	That any of this works with real permissions, the real Az.Quota module and real Azure APIs
E2E tests	None: no `quota-groups-management-e2e-test` folder, script or DR pipeline	Everything end to end

To run the unit tests locally, from the component folder with Pester installed:

```
Set-Location C:\src\Azure-Platform\components\quota-groups-management
Invoke-Pester .\scripts\Manage-AzQuotaGroup.Unit.Tests.ps1 -Output Detailed
```

E2E test approach

The E2E test runs the DevOps team's own functions against a temporary quota group that the test creates and deletes itself. It never touches the real quota groups in use.

Why build it

- The unit tests use fake Azure, so they can't catch broken permissions, expired service connections or Azure API changes.
- Az.Quota is an early module (0.2.0), so behaviour may change between versions.
- Without it, a break is found only when someone urgently needs quota and the pipeline fails.
- The other platform components have scheduled E2E tests; this one is the gap.

What each run creates, reuses and never touches

	What	Why
Creates	A temporary quota group with a unique test name, e.g. `e2e-qg-<runId>`	Safe to break; deleted after every run
Reuses	An approved test management group, test subscription(s) and service connection	Subscriptions can't be created in a test, and these already exist
Never touches	The real quota groups already in use, and PROD stages	Changing them could move real quota away from real subscriptions

Reading data (SKU names, current allocation) on the test subscription is safe, because it changes nothing.

What we borrow and what we build

Borrow	From	Used for
The engine functions (called, not copied)	`quota-groups-management/scripts/Manage-AzQuotaGroup.ps1`	Running each scenario
The environment file's structure	`environments/swissredev_com/exp/vars.yaml`	The test's own settings file
Module install template and pinned versions	`pipelines/templates/install-azquota-module.yml`, `build-meta.jsonc`	The same Az.Quota and Az.Accounts versions as the component
How each function is called	`Manage-AzQuotaGroup.Unit.Tests.ps1`	Reference for parameters
Folder layout, Pester `BeforeAll` / `AfterAll`, cleanup style	`central-tfstate-storage-e2e-test`	The test's structure
Pester pipeline template	`iam-access-governance-srz-e2e-test/pipelines/run-e2e-test-tmpl.yml`	Running the tests in a pipeline
Auth helpers `Get-Token`, `Get-OidcToken` (reuse, don't copy)	`e2e-test-shared/scripts/AuthenticationUtils.ps1`	Authentication, if needed

We build four things ourselves: the Pester test file with the scenarios, a `vars.yml` with test values, the pipeline with its schedule and cleanup step, and a README.

Calling style: the test dot-sources `Manage-AzQuotaGroup.ps1` and calls the functions directly. This is simpler than triggering the 7 component pipelines; pipeline-level tests can be added later with `PipelineUtils.ps1` (`RunPipeline`, `WaitForPipelineRunCompletion`).

Proposed folder layout

```
components/
  quota-groups-management-e2e-test/
    README.md
    scripts/
      QuotaGroups.EndToEnd.Tests.ps1
    environments/
      swissredev_com/exp/vars.yml
    pipelines/
      pipeline.yml
```

E2E scenarios

Thirteen scenarios, in the order one test run goes through them. Phase 1 (scenarios 1–7 and 10–12) is safe to build now; phase 2 previews quota moves; phase 3 needs approval.

#	Scenario	Steps	Expected result	Phase	Status
1	Create group	`Initialize-AzureQuotaGroup` with the unique test name	Group exists under the test management group (`Test-AzureQuotaGroupExists` = true)	1	Not started
2	Create again	Run the same command again	No error; still one group	1	Not started
3	Preview only	`Initialize-AzureQuotaGroup -WhatIf` with a new name	No group is created	1	Not started
4	Add member	`Add-AzureQuotaGroupSubscription` with the test subscription	Listed by `Get-AzureQuotaGroupSubscriptions`, state SUCCEEDED	1	Not started
5	Delete while not empty	`Remove-AzureQuotaGroup`	Refused; group still exists	1	Not started
6	Read SKU names	`Get-AzureComputeQuotaNames` for the test subscription and region	Returns a non-empty list	1	Not started
7	Read allocation	`Get-AzureQuotaGroupSubscriptionQuota` for subscription, region, SKU	Returns the allocation, or the classic-quota fallback	1	Not started
8	Preview a quota request	`Request-AzureQuota -WhatIf`, recipient = member, amount 13	Plan logged; amount rounded up to 20; allocation same before and after	2	Not started

#	Scenario	Steps	Expected result	Phase	Status
9	Non-member recipient	`Request-AzureQuota` for a subscription not in the group	Fails with a clear error; nothing changes	2	Not started
10	Remove member	`Remove-AzureQuotaGroupSubscription`	No longer listed as a member	1	Not started
11	Delete empty group	`Remove-AzureQuotaGroup`	Group is gone (`Test-AzureQuotaGroupExists` = false)	1	Not started
12	Cleanup always runs	Force a failure mid-run	Member removed and group deleted anyway; no group with the test prefix remains	1	Not started
13	Real small transfer (optional)	Request +10 of a cheap SKU between two approved test subscriptions, then reverse it	Recipient allocation rises by 10, then returns to its starting value	3	Not started

Not in E2E: syncing Enterprise Agreement subscriptions, because it could add real subscriptions to a group. At most, run it with `-WhatIf`.

Setup, cleanup and the E2E pipeline

Cleanup is the most important part: every run must leave nothing behind, even when a test fails or the run is cancelled.

Setup (before the tests)

1. Read the test management group, group-name prefix, test subscription and service connection from `vars.yml`, never from the code.
2. Install the pinned modules with the component's `install-azquota-module.yml` template.
3. Delete any leftover groups with the test prefix from earlier failed runs.
4. Generate a unique group name, e.g. `e2e-qg-<runId>`, and check the test subscription is the approved one.

Assertions

- Check real Azure state with the read functions (`Test-AzureQuotaGroupExists`, `Get-AzureQuotaGroupSubscriptions`), not log text.

Cleanup (after the tests)

1. Remove the test subscription from the group first, then delete the group. Only touch the uniquely named test group.
2. Use Pester `AfterAll`, and wrap the test run in `try/finally`, so cleanup runs after a failed assertion.
3. Add a separate cleanup step in the pipeline with `condition: always()`.

4. Keep short manual cleanup steps in the README for a hard cancellation or a lost agent.

```
# Skeleton only
try {
    # Create unique group; track created group and member IDs
    # Run the Pester scenarios against the test fixture
}
finally {
    # If a member was added: Remove-AzureQuotaGroupSubscription
    # If the test group exists and is empty: Remove-AzureQuotaGroup
}
```

The E2E pipeline

- `trigger: none` plus a schedule, so it doesn't run on every commit. Copilot suggested nightly at 02:00; weekly may be enough.
- Built from the existing Pester template (`run-e2e-test-tmpl.yml`).
- DEV (or prod-dev) only. It never runs the component's PROD stages.
- A separate DR pipeline is probably not needed: the component has no regional failover to test.

The test steps run in the order of the scenario table above.

Open decisions

Scenarios 1–12 can be built now; these decisions are needed before the test can run for real. Ferenc doesn't know this component well, so ask its author (find them in the git history or the original PR for `components/quota-groups-management`) or your lead.

- [ ] DEV tenant or prod-dev? (Copilot suggests DEV; the other E2E tests use prod-dev.)
- [ ] Which test management group and test subscription(s) are approved?
- [ ] Which service connection is allowed to create quota groups there?
- [ ] Are create, add and remove asynchronous? If so, how long should the test wait?
- [ ] Quota requests: WhatIf only, or a small real transfer (scenario 13)? How is a transfer reversed?
- [ ] Schedule: nightly or weekly?
- [ ] Is a DR pipeline needed?
- [ ] Who reviews the PR?